

tmux documentation

Per-Session Isolation Plan

Engineering plan for: per-tmux-session OOM containment, CPU + memory caps, and the matching anti-bluff regression coverage. Captures forensic evidence, design decisions, and the work items needed to ship this safely. **No implementation lands until the plan is approved.**

§0 — Problem statement (with captured evidence)

Two distinct bugs in the current architecture, both proven on the podman-machine VM 2026-05-13:

Bug 1 — tmx session shell is the VM's core user, not the operator's host shell

```
$ tmx new -s X
... operator now sees: core@localhost:~$
```

/proc/self/cgroup shows 0::/user.slice/user-501.slice/session-353.scope and pwd returns /var/home/core. The macOS host's commands are not on PATH; the operator is in a Linux user shell inside the podman machine VM.

Cause — by design: the verified Linux binary runs in the VM, and the bridge SSHes the operator's terminal into the VM where tmux runs.

Status — fundamental architectural limit. Mitigation, not "fix": the operator's /Users/<name> is virtiofs-mounted into the VM, so files ARE accessible. macOS-only commands (Homebrew binaries, `scutil`, etc.) cannot be invoked from inside a Linux session. We document this explicitly.

Bug 2 — All tmx sessions share ONE cgroup (the §1 isolation bluff)

```
$ tmx new -s isol1 -d
$ tmx new -s isol2 -d
$ pgrep -af tmux/build/bin/tmux
504655 /Users/.../tmux/build/bin/tmux new -s isol1 -d ← ONE server
$ cat /proc/504655/cgroup
0::/user.slice/user-501.slice/user@501.service/app.slice/run-p504653-i5046
                                     ← ONE scope
```

Both sessions live in **one tmux server**, that server's process is in **one cgroup scope**, with **one** MemoryMax and **one** TasksMax.

README claim: *"Crash isolation: if one session OOMs/crashes, only that session dies; other sessions and their processes survive."* — **this is false in practice.** OOM in any session OOM-kills the shared server, which destroys every session. Test 14

(`14_concurrent_oom_independence.sh`) passes because it tests three SEPARATE scopes spawned with explicit unit names — it does NOT test the actual `tmx new -s X / tmx new -s Y` operator workflow. **§1 / §11.4.1 FAIL-bluff in the test suite itself.**

§1 — Design

Each `tmx new -s NAME` invocation MUST result in:

1. A dedicated `cgroup-v2` transient scope named `tmx-<NAME>.scope` per the operator-given session name.
2. The session's primary shell process (and all descendants) is placed inside that scope.
3. Independent `MemoryMax`, `CPUQuota`, `TasksMax` per session — defaults from `env` (`TMX_MEM`, `TMX_CPU`), overridable per session.
4. OOM-kill in scope A leaves scope B + the `tmux` server + the `user.slice` untouched (§1 invariant).

Implementation approach — default-command injection

`Tmux` supports `set -g default-command "..."` and the `new-session -e VAR=val` flag, plus `new-window -c / -e`. The cleanest path:

```
tmux new-session -d -s NAME \  
    "systemd-run --user --scope --quiet \  
        --unit=tmx-NAME.scope \  
        -p MemoryMax=8G -p CPUQuota=200% -p TasksMax=4096 -p Delegate=yes \  
        --setenv=TMX_SESSION=NAME \  
        $SHELL -l"
```

The session's initial shell is the `systemd-run` command, which transitions into the user's `$SHELL -l` inside the new transient scope. Any process spawned from that shell (new windows, panes via `:new-window`, child processes) inherits the scope via Linux `cgroup-v2` semantics.

Per-window / per-pane creation: `tmux`'s `:new-window` and `:split-window` spawn children of the existing pane's process. Because `cgroup` inheritance is via `fork()`, they automatically join the session's scope. No extra wrapping needed.

Cross-session boundary: when the operator runs `:choose-tree` to jump between sessions, that's a `tmux`-internal client-side switch (re-attach); it doesn't move processes between scopes. Each session's process tree stays in its own scope.

Why NOT podman-per-session

The user's question mentions "the power of containers." We evaluated both:

Property	<code>systemd --user --scope</code>	podman per session
CPU + memory cap	yes (<code>cgroup v2</code>)	yes (<code>cgroup v2</code>)
OOM containment	yes — kernel kills only the scope	yes
Filesystem isolation	shared with user — host files visible	own rootfs
Network isolation	shared	own namespace
Startup latency	~80 ms	~1.5 s (image pull amortised)
State persistence on detach	yes (scope persists w/ <code>Delegate=yes</code>)	yes (container persists)
<code>/Users/<name></code> access	inherited from user	requires explicit mount

Property	<code>systemd --user --scope</code>	podman per session
Required infra	<code>systemd ≥ 230</code> (we already check this)	podman/docker on the host
Matches what OOM-Protect does	identical pattern	different

`systemd --user --scope` provides every property the user actually asked for ("CPU + memory cap", "kill session A leaves others alone"). Full OCI containers add filesystem + network isolation we do not need and trade off ~20× startup latency. **We adopt the OOM-Protect approach of per-leaf scopes**, applied per tmux session.

If, after this lands, the operator wants stronger isolation (separate rootfs, etc.) we can add `TMX_BACKEND=podman` as an opt-in alternative. Out of scope for this plan.

§2 — Anti-bluff regression coverage (§11.4 enforcement)

Three things must change so this CAN'T re-ship broken:

§2.1 Test 14 must exercise the operator's actual workflow

Current `14_concurrent_oom_independence.sh` spawns three explicit `systemd-run --user --scope` units by hand and triggers OOM in scope A. This passes EVEN when the actual `tmx new -s X / tmx new -s Y` flow puts all sessions in one cgroup — that was the gap that let Bug 2 ship.

Fix: rewrite Test 14's body to use the `tmx` wrapper to create the three sessions (`tmx new -s A -d, tmx new -s B -d, tmx new -s C -d`). Then trigger OOM in A's shell via `tmx send-keys -t A 'stress-ng ...'` and verify B + C are still listed in `tmx ls` AND their scope cgroup files still report their MainPID. Operator-facing path is now in the gate.

§2.2 New Test 15 — per-session cgroup distinctness

`15_per_session_cgroup_distinct.sh`:

Assertion	Positive evidence
T1: <code>tmx new -s a -d + tmx new -s b -d</code> produce distinct <code>systemctl --user list-units --type=scope</code> entries <code>tmx-a.scope + tmx-b.scope</code>	<code>systemctl --user is-active tmx-a.scope + tmx-b.scope</code> both = active
T2: each scope's <code>cgroup.procs</code> file contains its session's shell PID (different PIDs in different cgroup paths)	<code>/sys/fs/cgroup/.../tmx-a.scope/cgroup.procs</code> and <code>.../tmx-b.scope/cgroup.procs</code>
T3: each scope's <code>memory.max</code> reads back the operator-configured cap (default <code>TMX_MEM=8G</code>) — values are 8 589 934 592 bytes	direct readback
T4: each scope's <code>cpu.max</code> reads back the operator-configured quota	direct readback
T5: <code>TMX_MEM=2G tmx new -s small -d</code> produces a scope whose <code>memory.max = 2 147 483 648</code>	direct readback

Assertion

T6: cgroup paths are operator-namespaced (tmx-a.scope not tmx-<random>.scope) so operators can target by name

Positive evidence

string compare on ControlGroup output

§2.3 New Test 16 (destructive, TMX_TEST_DESTRUCTIVE=1) — OOM containment is genuine

16_per_session_oom_containment.sh:

Assertion

T1: 3 sessions A/B/C created via `tmx new -s X -d`, all visible in `tmx ls`

T2: capture B's MainPID and C's MainPID from `cgroup.procs`

T3: trigger OOM in A — `tmx send-keys -t A "stress-ng --vm 1 --vm-bytes <MemoryMax+10%> --timeout 30s"`

T4: scope A is inactive after OOM

T5: scopes B and C are STILL ACTIVE

T6: B's MainPID and C's MainPID UNCHANGED

T7: tmux SERVER still alive (`tmx ls` still returns B + C)

T8: user.slice survived (default.target = active throughout)

Positive evidence

tmx ls output

direct readback

journalctl -k oom-kill line

`systemctl --user is-active tmx-A.scope = inactive`

`systemctl is-active = active`

re-read `cgroup.procs`

process and listing both confirm

`systemctl is-active`

§2.4 New meta-test mutations M9 + M10

Mutation

M9: strip the per-session scope wrap from `tmx.template` (revert to one-server-one-scope)

M10: hardcode `MemoryMax=infinity` in the per-session scope

Behaviour

sessions share a cgroup

memory cap not enforced

Must be caught by

Test 15 T1 (only one scope appears)

Test 15 T3 (memory.max ≠ 8G)

§2.5 e2e test extension

`scripts/test_e2e.sh` adds T7:

- T7: create two sessions through the bridge, verify both are in different scope units via `tmx run-shell "systemctl --user list-units --type=scope"` output (or equivalent). End-to-end through bridge → wrapper → systemd.

§3 — Files to change

File

`scripts/tmx.template`

Change

Per-session `--unit=tmx-<NAME>.scope` wrapping in the new-session branch; share-or-

File

scripts/tests/
14_concurrent_oom_independence.sh

scripts/tests/
15_per_session_cgroup_distinct.sh
scripts/tests/
16_per_session_oom_containment.sh
scripts/tests/
meta_test_false_positive_proof.sh

scripts/challenges/tmux.yaml

scripts/test_e2e.sh
scripts/tmx-mac.template

docs/plans/containerization.md

docs/guide/README.md

README.md

Constitution.md
CLAUDE.md / AGENTS.md
Containers/CONSTITUTION.md / Containers/
CLAUDE.md / Containers/AGENTS.md

Change

isolate decision tree; sanitise NAME to a unit-name-safe token
Replace hand-spawned scopes with `tmx new -s` invocations so the operator path is what's tested

New — §2.2

New — §2.3 (destructive, opt-in)

New M9 + M10 — §2.4

New CH-15 + CH-16 entries; CH-14 updated to reflect operator-path coverage
New T7 — §2.5
No change — bridge already delegates to VM-side wrapper
Update §B-conclusion to "per-session scope (NOT per-server scope)"
New §5.6 explaining per-session isolation + how to override caps via `env`
Architecture diagram update — show per-session scope tree, not single-shared scope
New §11.4.7 — operator-path test coverage rule (see §4 below)
Reflect §11.4.7 + new test commands
Propagate §11.4.7 (anti-bluff in submodule governance per user mandate)

§4 — Constitution amendment §11.4.7

Add a new clause that closes the exact gap that let Bug 2 ship:

§11.4.7 — Operator-path test coverage rule. Every gate test for a feature **MUST** exercise the SAME entry point an end-user would invoke in production. Tests that bypass the operator's wrapper, helper, or install path — and instead reproduce its effects with hand-crafted equivalents — **DO NOT** satisfy §11.4.2's captured-evidence requirement. When the operator's path and the test's path diverge, the test document must **EXPLICITLY** name what divergence exists **AND** a separate end-to-end test must close that divergence with captured evidence.

Forensic anchor: Test 14 hand-spawned `systemd-run --user --scope` units by hand to simulate isolation, while the actual `tmx new -s X` operator workflow placed every session in one shared scope. Test 14 reported GREEN; operator-facing isolation did not exist. Caught 2026-05-13 (see Fixed.md). Propagated to Containers/CONSTITUTION.md at the same anchor depth.

§5 — Acceptance criteria

This plan is "done" when:

1. `bash scripts/test_vm.sh` → SUMMARY PASS=16 (added Test 15 + Test 16) FAIL=0
2. `TMX_TEST_DESTRUCTIVE=1 bash scripts/test_vm.sh` → SUMMARY PASS=16 FAIL=0 SKIP=0
3. `META=1 bash scripts/test_vm.sh` → 10 mutations caught (M1..M10) + reverted
4. `bash scripts/test_e2e.sh` → PASS=9 (added T7) FAIL=0
5. Manual operator check: `tmx new -s A -d; tmx new -s B -d; systemctl --user list-units --type=scope` shows `tmx-A.scope` AND `tmx-B.scope` AS DISTINCT UNITS
6. Manual operator OOM check (Linux host with destructive opt-in): kill session A's shell with a stress-ng exhausting `MemoryMax+10%`; session B's tmux pane still responds to `tmx send-keys -t B "echo alive" + tmx capture-pane -t B -p` shows `alive`
7. Constitution.md §11.4.7 + Containers/CONSTITUTION.md equivalent both reference the same forensic anchor

§6 — Operator decisions (2026-05-13)

Answered by operator before implementation:

1. **Default TMX_MEM per session — host-adaptive (§12.6 budget).** Compute at session-create:

$$\text{MEM_MB} = \max(\text{MemTotal_MB} * 0.6 / 4, 2048)$$

Honours Constitution §12.6 ($\Sigma(\text{active TMX_MEM}) \leq 0.6 \times \text{MemTotal}$) with an assumed concurrent-session budget of 4. Operator override via `TMX_MEM=12G` `tmx new -s heavy` always wins. CPU stays 200% per session (oversubscription is acceptable; kernel time-slices).
2. **Unit-name sanitisation — replace invalid chars with `_`, error on collision.** `tmx new -s 'my work'` → `tmx-my_work.scope`. If `tmx-my_work.scope` already exists (active), `tmx new` errors explicitly with the path so the operator can diagnose. Predictable naming — operators can target a scope by its session name.
3. **`tmx kill-session cleanup` — explicit `systemctl --user stop tmx-NAME.scope` AFTER tmux's own kill-session.** Belt-and-suspenders: faster cgroup-number / memory-accounting reclaim; tolerant of logind versions that GC inactive scopes slowly.
4. **macOS host-shell access — document explicitly, no bridging.** `docs/guide/README.md` §5.6: "tmx sessions are Linux environments running in the podman machine VM. Your files under `/Users/<name>` are accessible via virtiofs; macOS-only Mach-O commands (Homebrew binaries, `scutil`, etc.) are not available inside a session." Operators use system tmux (Homebrew) for macOS-host work; our verified tmux is for the bounded-isolation workflows. Side-by-side is already the design.